RECYCLEGPT: AN AUTOREGRESSIVE LANGUAGE MODEL WITH RECYCLABLE MODULE

Yufan Jiang*, **Qiaozhi He***, **Xiaomin Zhuang**, **Zhihua Wu**,
Kunpeng Wang[1], **Wenlai Zhao**[1], **Guangwen Yang**[1]
[1]Department of Computer Science and Technology, Tsinghua University, Beijing, China

arXiv:2308.03421v3 [cs.CL] 23 May 2024

ABSTRACT

Existing large language models have to run K times to generate a sequence of K tokens. In this paper, we present RecycleGPT, a generative language model with fast decoding speed by recycling pre-generated model states without running the whole model in multiple steps. Our approach relies on the observation that adjacent tokens in a sequence usually have strong correlations and the next token in a sequence can be reasonably guessed or inferred based on the preceding ones. Experiments and analysis demonstrate the effectiveness of our approach in lowering inference latency, achieving up to 1.4x speedup while preserving high performance.

1 INTRODUCTION

Large language models (LLMs) (Brown et al., 2020; OpenAI, 2023; Touvron et al., 2023; Chowdhery et al., 2022; Biderman et al., 2023; Smith et al., 2022) have revolutionized the field of natural language generation for their abilities in generating satisfactory text across various application domains. The excellent performance benefits greatly from the scaling of model size (100B+ parameters), but at the same time, the fact remains that a single decoding step gets slower as the model gets larger.

In addition to the immense computation introduced by larger models, a larger memory footprint is also a major factor causing slower inference of LLMs (Dao et al., 2022; Pope et al., 2023). This large memory footprint includes the trained model parameters, the temporary state used during inference, and in addition to these, the KV cache is also stored in memory. At each decoding step, it has to load the parameters and KV cache from high-bandwidth memory (HBM) into the compute cores which results in significant memory traffic and thus, high total memory bandwidth is required to meet a given latency target. In other words, the speed of generating tokens in LLMs is primarily limited by how fast it can access memory (Shazeer, 2019; Pope et al., 2023; Chen et al., 2023). And the time to generate each token is roughly proportional to the number of model parameters. Since each new token generated by the model depends on the previous tokens, many calls to the transformer model are necessary to generate an entire sequence.

To make inference more efficient, several works are proposed. The core idea of these works is how to reduce the memory footprint and alleviate memory traffic problems. For example, distillation (Hinton et al., 2015), sparcification (Jaszczur et al., 2021), quantization (Shen et al., 2020; Zafrir et al., 2019) and sharing weights (Xiao et al., 2019; Zeng et al., 2021) are proposed to reduce the model size. Adaptive computation (Sukhbaatar et al., 2019; Schwartz et al., 2020) aims to use fewer computing resources for easier inference steps. Multi-Query Attention (Shazeer, 2019; Ainslie et al., 2023) shares the keys and the values to reduce the size memory bandwidth requirements while Flash Attention (Dao et al., 2022) uses a small amount of computation to reduce the number of memory reads/writes. Though the above works propose effective approaches, they usually require changing the model architecture or attention algorithm, adding more training tasks, and re-training these complicated models. Recently speculative decoding methods have become popular (Leviathan et al., 2023; Chen et al., 2023; Miao et al., 2023). To reduce the number of executions of the large model, they employ a two-step approach: first, an efficient small model speculatively generates the

*Equal contribution, correspondence to {jiangyufan2018,qiaozhihe2022}@outlook.com

simpler parts of the text; then, a large model is used to validate those parts, rather than having the large model generate the entire text alone. This idea is simple and convenient and also has been integrated to open-source frameworks. However, the selection of efficient models is still an open question. Using the small version of LLMs may be one solution while it still needs sequence-level distillation.

Naturally, adjacent tokens in a sequence have strong correlations. That is to say, in many cases, the next token in a sequence can be reasonably guessed or inferred based on the preceding ones. This phenomenon leads us to investigate an efficient decoding method in another research direction, with the goal of generating as many tokens as possible under the same amount of memory processing budget. We propose **RecycleGPT**, a novel language model architecture that is inherently capable of fast decoding by recycling pre-generated model states. In our approach, we modify the original language model by adding an additional recyclable module that predicts the next several tokens using previously generated states without running the entire model multiple times, which can also be viewed as a recycling process. The recyclable module is made up of a stack of transformer-based layers for achieving more efficient representations to make predictions. During inference, this module can be used with the standard language model decoding pipeline in various ways. In this paper, we choose to use them alternately (i.e., generating every two tokens requires running the complete model once) and leave exploring more strategies for future work. Despite its simple architecture, the recyclable module can effectively represent contextual information and make accurate predictions, thereby achieving the goal of accelerating the decoding process.

We evaluate the RecycleGPT on a set of standard benchmarks. It achieves a 1.4x speedup over the standard language model, yet with no loss in performance. More importantly, it is orthogonal to previous methods and is straightforwardly applicable to different LLMs. The main contributions of this work are summarized as follows:

- We propose a novel generative language model RecycleGPT and release RecycleGPT-1.3B. Compared to standard language models, our model achieves 1.4x speedup with only 15% extra parameters introduced, while maintaining comparable performance on downstream tasks. In the future, we will release variants of RecycleGPT in different sizes.

- Our recycling method is flexible and scalable, which can be applied to different pre-trained models. Moreover, the size of the recyclable modules and the generation strategy can be adjusted to achieve the desired speedup performance.

2 Background

In this section, we provide some background on the memory cost at inference time. We also give a brief introduction to the auto-regressive language model.

2.1 Inference Memory Cost

As the model scale continues to explode exponentially, language model decoding becomes highly costly and inefficient. Except that larger models introduce more tensor computations that take up a certain amount of time, the memory transfer also occupies a significant portion of time. Generally, large language models have a large memory footprint for storing both model parameters and KV cache which are usually stored in on-device high-bandwidth memory (HBM). These tensors need to be transferred from HBM to the compute cores each forward pass which takes a certain amount of time. And since the auto-regressive language models generate one token each step until the end symbol is reached, many calls to the language model are necessary to generate an entire sequence. According to Pope et al. (2023), at small batch sizes and sequence lengths, loading weights takes the most time, while loading the KV cache dominates inference time at a large scale. Moreover, larger language models need multiple devices to work together in parallel, which also adds communication overhead. Thus, how to reduce the memory size and transfer frequency is another key factor in accelerating the model decoding process.

2.2 AUTO-REGRESSIVE LANGUAGE MODEL

Given a corpus of tokens $X = \{x_1, ..., x_n\}$, an auto-regressive language model (Figure 1 (a)) factors the joint probability into a chain of conditional probabilities with a left to right causal structure:

$$\mathcal{P}_{\mathcal{AR}}(X; \theta_{AR}) = \prod_{i=1}^{n} p(x_i | x_{<i}; \theta_{AR}), \tag{1}$$

For most LLMs, transformer-based models are used to capture the above causal structure of the output distribution. Generally, in transformer, there are L identical stacked layers. Each of them is composed of a self-attention sub-layer and a feed-forward sub-layer (FFN). Both of them are equipped with a residual connection and a layer normalization unit. For more details, we refer the reader to Vaswani et al. (2017). When generating the token x_{t+1}, a distribution over vocabulary tokens is computed via a softmax-normalized linear classifier $\mathbf{W}_L$ with h_t^L as input:

$$p(x_{t+1} | h_t^L) = \text{softmax}(\mathbf{W}_L h_t^L), \tag{2}$$

where h_t^L is the decoder state of the last layer of the transformer model. Finally, the (greedily chosen) prediction x_{t+1} can be written as:

$$x_{t+1} = \text{argmax}\, p(x_{t+1} | h_t^L) \tag{3}$$

At the same time, maximum likelihood training with a cross-entropy loss can be applied at each decoding step:

$$\mathcal{L}_1 = \log \mathcal{P}_{\mathcal{AR}}(X; \theta_{AR}) = \sum_{i=1}^{n} \log p(x_i | x_{<i}; \theta_{AR}), \tag{4}$$

Though the transformer structure shows strong generative capabilities and high parallelism during training. It has been pointed out that the auto-regressive format is highly memory bandwidth bound and is difficult to leverage modern accelerator hardware effectively (Chen et al., 2023; Shazeer, 2019). This kind of memory-bound model generates one word per call, hence generating multiple words in sequence induces high latency and it gets worse as the number of model parameters increases.

3 RECYCLEGPT

In order to minimize the time spent on both memory transfer and computation, we aim to reduce the number of calls of the full-parameter language model. Instead of always making predictions according to the previous token, we propose a simple but effective solution. Based on the assumption that neighboring tokens are highly correlated and interdependent, we directly recycle the representation of the current token to predict the following m consecutive tokens without feeding each predicted token into the language model step by step. In this work, we only focus on the case where m is 2 and we leave exploring this for future work.

Thus we introduce RecycleGPT, a new generative language model. Figure 1 shows the overall framework. RecycleGPT includes a simple but effective recyclable module that is made up of a stack of N identical transformer layers. We use these few layers to predict the next token directly without feeding the current hidden state to the bottom of the language model and run the whole model to make predictions. The design of these layers should consider how to strengthen the dependencies between discontinuous tokens, i.e. two tokens with one space and we will give a detailed introduction in the next section. When generating token x_{t+1}, decoder state h_{t-1}^L and embedding e_t of token x_t are passed through the recyclable module (Recycle) to obtain alternation state h'_t which can be fed into linear classifier layer to predict x_{t+1} like Eq.(2) and Eq.(3):

$$\begin{aligned} x_{t+1} &= \text{argmax}\, p(x_{t+1} | h'_t), \\ p(x_{t+1} | h'_t) &= \text{softmax}(\mathbf{W}_L h'_t), \\ h'_t &= \text{Recycle}(g(h_{t-1}^L, e_t)), \end{aligned} \tag{5}$$

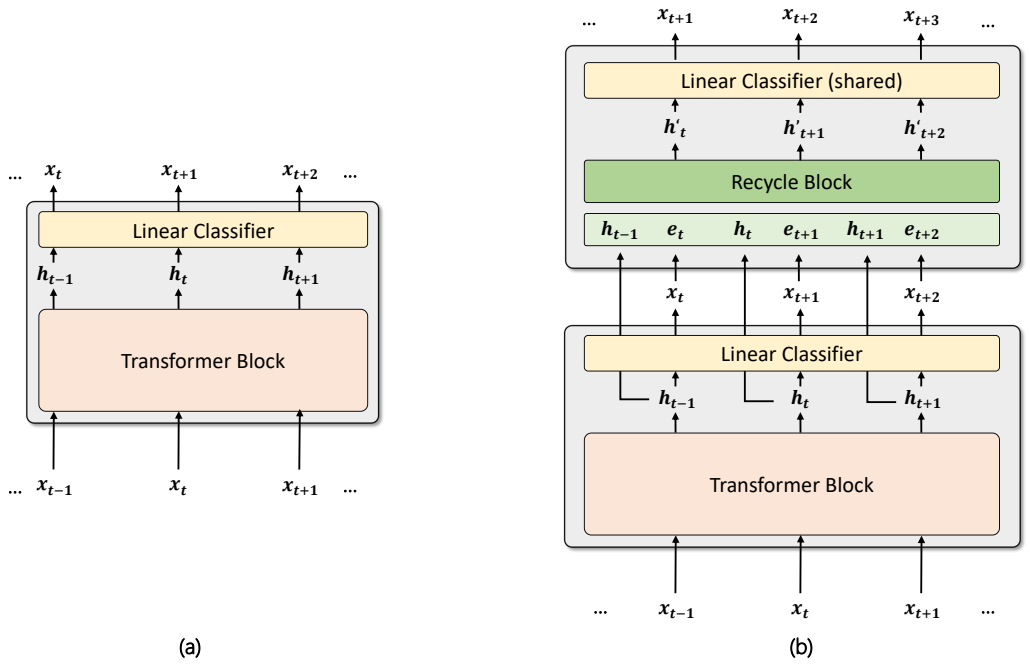

Figure 1: **Model architecture of standard GPT and RecycleGPT**.

where $g(\cdot,\cdot)$ is the function to integrate two streams of representations. We adopt the concatenating method for combining these two representations which is also introduced in the next section. According to 5, we use the following objective to optimize the parameters of Recycle:

$$\mathcal{L}_2 = \log\mathcal{P}_{Recycle}(X;\theta_{Recycle}) = \sum_{i=1}^{n} \log p(x_i|x_{<i-1};\theta_{Recycle}), \tag{6}$$

In this work, we build RecycleGPT, a transformer based language model with a recyclable module, and train it from scratch. Thus, the training objective of our language model can be formulated as:

$$\mathcal{L}_3(X) = \mathcal{L}_1(X) + \lambda * \mathcal{L}_2(X), \tag{7}$$

Where λ is a hyper-parameter to balance the effect of each loss term.

For easier understanding, we illustrate the difference between auto-regressive decoding and our methods in Figure 2. Rather than generating h_t^L through the complete execution of the language model using token x_t as the input. We generate h'_t by the recyclable module with the hidden state of the last step and the token it predicted. After obtaining h'_t, we can directly use it to predict token x_{t+1}. Recycle module can speed up decoding due to its compact structure compared with whole language model layers. Based on the m being set to 2 in this work, we adopt a simple strategy of alternately using h'_t and h_t^L to generate the next word for each inference step. Moreover, Our RecycleGPT can also perform standard auto-regressive decoding without using the recyclable module which we denote as RecycleGPT-std in the experiments section.

3.1 Recyclable Module

In this section, we give a detailed description of the Recyclable module. This module is introduced to generate the substitute for the original decoder state h_t^L which can be used to predict the next token. The recyclable module helps the language model exploit the dependencies between discontinuous words. There are various ways to construct this module such as GRU (Cho et al., 2014), LSTM (Graves & Graves, 2012), and FFN (Vaswani et al., 2017). In this paper, we employ a number of

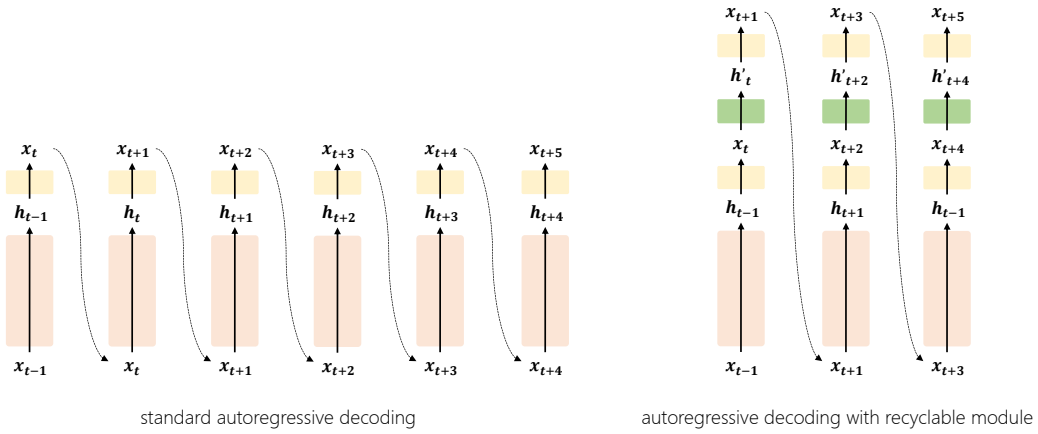

Figure 2: **Illustration of the difference between standard autoregressive decoding and autoregressive decoding using a recyclable module.**. The orange block indicates one forward call of the whole language model while the green one indicates the call of the recyclable module. The amount of computation and memory footprint required by the green part is far less than that of the orange part. When using an alternating decoding strategy, we see that the recyclable module can save a significant amount of time. The yellow block indicates the final output classifier.

transformer layers (Vaswani et al., 2017) for better capturing the relationship between discontinuous tokens. Recently, there are many variants of transformer layer, and we choose LLaMA, (Touvron et al., 2023) a stronger one among them. It leverages various improvements that are subsequently proposed, or used in different large language models, like RMSNorm (Zhang & Sennrich, 2019), swiGLU activation function (Shazeer, 2020) and rotary embeddings (Su et al., 2021).

Figure 1 depicts the structure of the recyclable module. Before fed into the recyclable module, h'_{t-1} and e_t are concatenated along the length dimension at first. And we also set position embeddings for them. Given the merged sequence $\{h_0, e_1, h_1, e_2..., h_t, e_{t+1}, h_{t+1}, e_{t+2}\}$, the corresponding position embedding is set to $\{0, 1, 1, 2..., t, t+1, t+1, t+2\}$ for both standard absolute position embeddings and rotary embeddings. Then, the concatenation of two representations is passed through a stack of N pre-norm LLaMA layers (Wang et al., 2019; Touvron et al., 2023) which consist of self-attention sub-layers and feed-forward sub-layers to get the final representation of recyclable module. The number of recyclable module layers N in this work is adjustable based on hardware constraints to achieve the desired speedup performance. For example, when N is set to 6, the recyclable module introduces approximately 15% extra parameters and achieved a 40% decoding speedup when using the alternating decoding strategy. Compared to other methods that reduce the number of model invocations, such as speculative decoding (Chen et al., 2023; Leviathan et al., 2023), our method is fine-grained while also being orthogonal to their methods, allowing further acceleration on top of them.

4 EXPERIMENTS

4.1 EXPERIMENTAL SETUPS

Training Data. Our model is trained on the Pile (Gao et al., 2020; Biderman et al., 2022), a carefully selected group of English language datasets for training large language models. The Pile is well-suited for training large autoregressive transformers. The reason we choose this public dataset is that it can achieve higher downstream performance than other popular datasets like C4 (Raffel et al., 2020) and OSCAR (Suárez et al., 2019). Additionally, this dataset has been widely utilized by state-of-the-art models including GPT-NeoX-20B (Black et al., 2022), Megatron-Turing NLG 530B (Smith et al., 2022), OPT (Zhang et al., 2022) and Pythia (Biderman et al., 2023). We use the BPE tokenizer developed by Touvron et al. (2023). Overall, our entire training dataset contains 360B tokens after tokenization.

Training. We select LLaMA (Touvron et al., 2023) as our backbone and train a 1.3 billion parameter model. The RecycleGPT has 24 layers with 2048 hidden units and 32 attention heads. We set $N = 6$ for the recyclable module and it introduces 15% parameters to the original model respectively. λ is set to 1 in this work. Our model is trained using the Adam optimizer with the following hyper-parameters: $\beta_1 = 0.9$, $\beta_2 = 0.95$. Inspired by some of the latest research works (Biderman et al., 2023; Brown et al., 2020), we use a larger batch size than the standard language model. As a result, we use a batch size of 1280 samples, with each sample having a sequence length of 2048 tokens for our model. The detail of the pre-training settings can be found in Appendix 4. When using RecycleGPT for decoding, we can choose to use the recyclable module for alternating generation denoted as RecycleGPT-rec, or perform standard auto-regressive decoding denoted as RecycleGPT-std.

We adopt several efficient implementations to improve training speed. First, we use flash attention (Dao et al., 2022) during training to increase device throughput. In addition, we leverage the Zero Redundancy optimizer (ZERO) (Rajbhandari et al., 2020) to efficiently scale across multi-machine. We also use data parallelism (Goyal et al., 2017) and tensor parallelism (Shoeybi et al., 2019) to optimize performance.

Evaluation. We empirically evaluate RecycleGPT on several common language modeling benchmarks in both zero-shot and few-shot settings.

- **Zero-Shot**. we provide the model with a textual description of the task and a test example as context. The model is then tasked with either generating an open-ended answer or ranking a set of multiple-choice answers.
- **Few-Shot**. we provide the model with a few examples of the task and a test example as context. The model is then tasked with either generating an open-ended answer or ranking a set of multiple-choice answers.

We use the Language Model Evaluation Harness (Gao et al., 2021) to run evaluations and use the same evaluation metric with Biderman et al. (2023) for a fair comparison. Our efficiency metric is the speedup of the whole model for generating the full sequence with different lengths. We perform decoding on a single A100 GPU with 200 examples and the results come from the average of 3 individual runs. When decoding we use the greedy search method.

Baselines. For a fair comparison, we collected existing open-source language models with around 1.3B parameters as baselines that are listed below: 1) OPT (Zhang et al., 2022), a suite of decoder-only pre-trained transformers ranging from 125M to 175B parameters, and the architecture, tokenizer is almost identical to the standard GPT model. 2) Pythia (Biderman et al., 2023) a suite of LLMs all trained on Pile datasets ranging in size from 70M to 12B parameters. Pythia improve the original architecture with a few notable deviations based on recent advances in best practices for large-scale language models. Since the LLaMA (Touvron et al., 2023) did not release a 1.3B parameter baseline, we revisit a llama-1.3B ourselves using the pile dataset.

4.2 RESULTS

Common Sense Reasoning. We evaluate our models on standard common sense reasoning benchmarks, namely PIQA (Bisk et al., 2020), WinoGrande (Sakaguchi et al., 2021), ARC easy and challenge (Clark et al., 2018), SciQ (Welbl et al., 2017), LogiQA (Liu et al., 2020) and Lambada Storks et al. (2019) in the zero-shot setting.

In table 1, we report performance on six common sense reasoning benchmarks. On these benchmarks, our self-trained model and reproduced baseline model achieved competitive results with existing open-source models of the same size. The performance gap on some benchmarks may be caused by the differences in training data and the tokenizer we used. Compared to our own baseline, RecycleGPT using a standard decoding strategy (RecycleGPT-std) achieved comparable results, which proves that our recyclable module does not degrade the language model performance. Meanwhile, using the alternating decoding strategy (RecycleGPT-rec) can achieve 1.4x decoding acceleration with only less than one percentage point performance drop. In actual use, the decoding strategy can be chosen based on acceleration requirements. We will also provide more combinations such as multiple decoding strategies and different recyclable module sizes for selection in the future.

Model		PIQA	ARC-c	ARC-e	WinoGrande	Lambada	SciQ	LogiQA	Avg
OPT †	1.3B	71.7	23.7	57	59.7	57.9	84.5	22.3	53.8
Pythia †	1.4B	70.5	25.3	59.4	56	59.2	87.3	22.4	54.3
OPT	1.3B	71.6	23.3	57.2	59.2	57.9	84.3	22.4	53.7
Pythia	1.4B	70.8	26.0	60.6	57.3	61.7	86.6	21.2	54.9
GPT-Neo	2.7B	72.2	27.6	61.1	58.0	62.2	89.2	19.7	55.7
LLaMA-ours	1.3B	70.2	24.5	56.9	54.8	58.0	85.2	20.9	52.9
RecycleGPT-std	1.3B	70.6	25.0	57.1	55.4	58.1	87.5	20.7	53.5
RecycleGPT-rec	1.5B	68.7	24.6	56.7	55.3	57.6	86.4	23.8	53.3

Table 1: **Zero-shot performance on Common Sense Reasoning tasks.** Models with † denote that we directly report the scores from the Pythia paper Biderman et al. (2023), and others are from our implementation. Due to introducing the recyclable module, the number of parameters in our RecycleGPT has become 1.5B.

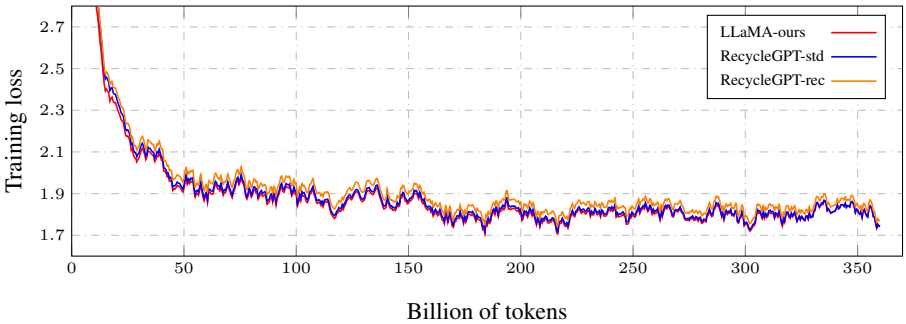

Figure 3: Training loss over train tokens.

Massive Multitask Language Understanding. We also evaluate our models on the massive multitask language understanding benchmark (MMLU) (Hendrycks et al., 2020) which consists of multiple-choice questions covering diverse domains of knowledge, such as humanities, STEM, and social sciences. At evaluation time, we use the examples provided by the benchmark, and the results of our models on the MMLU benchmark are reported in Table 2.

On this benchmark, RecycleGPT-1.3B outperforms OPT-1.3B and Pythia-1.4B and is Slightly lower than GPT-Neo-2.7B due to parameter size. Compared with the zero-shot setting, our RecycleGPT can achieve better results on the few-shot setting. A potential explanation is that our method is more applicable to situations with more examples or demonstrations due to the model architecture and decoding strategy we designed. Or perhaps our approach can better model certain types of context. This phenomenon also guides us on how to better utilize and improve our methods in the future. The detailed performance results on the 57 tasks of MMLU can be found in Table 5 in the appendix.

Figure 3 plots the training loss of the baseline, RecycleGPT-std, and RecycleGPT-rec. We can see that the training loss of baseline and RecycleGPT-std are almost identical which proves that our approach does not impact the performance of the original language model. At the same time, we also see that the curves of RecycleGPT-rec and baseline are very close. It demonstrates the effectiveness of our method. We report the speed (ms/token) of our RecycleGPT in table 3. RecycleGPT achieves a 1.4x speedup over the baseline model with KV cache and a 1.34x speedup without KV cache. The experiments in the current work were conducted on a 1.3B model due to computational constraints. In future work, we will experiment on larger models, such as 7B and 13B.

5 Related Work

The scale of auto-regressive language models grows from 117M (Radford et al., 2018) parameters to over 500B parameters (Smith et al., 2022) and various approaches are explored to improve the

Model		Humanities	STEM	Social Sciences	Other	Average
OPT	1.3B	22.8	25.7	23.3	26.5	24.6
Pythia	1.4B	26.6	25.6	24.3	26.6	25.8
GPT-Neo	2.7B	25.3	25.6	27.5	27.4	26.4
LLaMA-ours	1.3B	27.8	26.1	23.5	23.7	25.4
RecycleGPT-std	1.3B	26.5	28.2	24.0	25.0	26.2
RecycleGPT-rec	1.5B	26.3	28.0	24.0	24.8	26.0

Table 2: **Five-shot performance on Massive Multitask Language Understanding (MMLU).**

Model	ms/token					Avg	Avg Speed Up
	64	128	256	512	1024		
KV cache							
RecycleGPT-std	18.4	19.2	18.7	18.5	18.6	18.7	1X
RecycleGPT-rec	13.8	13.1	13.4	13.0	13.7	13.4	1.40X
w/o KV cache							
RecycleGPT-std	20.8	24.1	33.0	55.3	103.7	47.4	1X
RecycleGPT-rec	14.8	16.6	24.4	41.4	80.4	35.5	1.34X

Table 3: **Decoding speed of RecycleGPT-std and RecycleGPT-rec at different sequence lengths**.

inference efficiency. Large amounts of model computations and memory movements are the key factors of slower inference (Pope et al., 2023). To make model size smaller, several works are proposed distillation (Hinton et al., 2015; Sanh et al., 2019), pruning (Li et al., 2020; Brix et al., 2020; Zhou et al., 2021), sharing weights (Xiao et al., 2019) or quantization to int8 or even int4 (Dettmers et al., 2022; Shen et al., 2020; Zafrir et al., 2019;?). Adaptive computations (Sukhbaatar et al., 2019; Schwartz et al., 2020) try to reduce the amount of computation for easier inference steps. Sukhbaatar et al. (2019); Kitaev et al. (2020); Zeng et al. (2021); Roy et al. (2021); Choromanski et al. (2020) propose efficient attention layers to overcome the computational bottlenecks that time and memory scales quadratic in the sequence length. Based on the memory complexity of self-attention layers, Dao et al. (2022); Shazeer (2019) propose new attention algorithms to reduce the number of memory reads/writes between (HBM) and GPU on-chip SRAM.

Apart from improving the model architecture for faster decoding, sampling strategies, and partitioning strategies can also achieve low-latency inference (Stern et al., 2018; Ge et al., 2022). Speculative sampling methods employ multiple small efficient models to generate draft tokens and thus, run fewer forward calls of large model (Chen et al., 2023; Leviathan et al., 2023; Miao et al., 2023). For larger models that fit on different accelerator chips, practical partitioning approaches are proposed for balance workloads (Pope et al., 2023). This work also tries to minimize the number of forward calls of language models. Compared to previous methods that reduce the number of model invocations, such as speculative decoding (Chen et al., 2023; Leviathan et al., 2023), our method is fine-grained while also being orthogonal to their methods, allowing further acceleration on top of them.

6 Conclusion

In this work, we propose RecycleGPT, a new architecture with low-inference latency. By predicting multiple tokens with the recyclable module at once, RecycleGPT can achieve up to 1.4x speedup with no performance loss. The proposed approach is model-agnostic and complementary to previous acceleration techniques. In the future, we will explore more decoding strategies by combining the recyclable module and the original model in various ways.

REFERENCES

Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. Gqa: Training generalized multi-query transformer models from multi-head checkpoints. *arXiv preprint arXiv:2305.13245*, 2023.

Stella Biderman, Kieran Bicheno, and Leo Gao. Datasheet for the pile. *arXiv preprint arXiv:2201.07311*, 2022.

Stella Biderman, Hailey Schoelkopf, Quentin Gregory Anthony, Herbie Bradley, Kyle O'Brien, Eric Hallahan, Mohammad Aflah Khan, Shivanshu Purohit, USVSN Sai Prashanth, Edward Raff, et al. Pythia: A suite for analyzing large language models across training and scaling. In *International Conference on Machine Learning*, pp. 2397–2430. PMLR, 2023.

Yonatan Bisk, Rowan Zellers, Jianfeng Gao, Yejin Choi, et al. Piqa: Reasoning about physical commonsense in natural language. In *Proceedings of the AAAI conference on artificial intelligence*, volume 34, pp. 7432–7439, 2020.

Sid Black, Stella Biderman, Eric Hallahan, Quentin Anthony, Leo Gao, Laurence Golding, Horace He, Connor Leahy, Kyle McDonell, Jason Phang, et al. Gpt-neox-20b: An open-source autoregressive language model. *arXiv preprint arXiv:2204.06745*, 2022.

Christopher Brix, Parnia Bahar, and Hermann Ney. Successfully applying the stabilized lottery ticket hypothesis to the transformer architecture. *arXiv preprint arXiv:2005.03454*, 2020.

Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.

Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*, 2023.

Kyunghyun Cho, Bart Van Merriënboer, Dzmitry Bahdanau, and Yoshua Bengio. On the properties of neural machine translation: Encoder-decoder approaches. *arXiv preprint arXiv:1409.1259*, 2014.

Krzysztof Choromanski, Valerii Likhosherstov, David Dohan, Xingyou Song, Andreea Gane, Tamas Sarlos, Peter Hawkins, Jared Davis, Afroz Mohiuddin, Lukasz Kaiser, et al. Rethinking attention with performers. *arXiv preprint arXiv:2009.14794*, 2020.

Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, et al. Palm: Scaling language modeling with pathways. *arXiv preprint arXiv:2204.02311*, 2022.

Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? try arc, the ai2 reasoning challenge. *arXiv preprint arXiv:1803.05457*, 2018.

Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in Neural Information Processing Systems*, 35:16344–16359, 2022.

Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. Llm. int8 (): 8-bit matrix multiplication for transformers at scale. *arXiv preprint arXiv:2208.07339*, 2022.

Leo Gao, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, Charles Foster, Jason Phang, Horace He, Anish Thite, Noa Nabeshima, et al. The pile: An 800gb dataset of diverse text for language modeling. *arXiv preprint arXiv:2101.00027*, 2020.

Leo Gao, Jonathan Tow, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, Laurence Golding, Jeffrey Hsu, Kyle McDonell, Niklas Muennighoff, et al. A framework for few-shot language model evaluation. *Version v0. 0.1. Sept*, 2021.

Tao Ge, Heming Xia, Xin Sun, Si-Qing Chen, and Furu Wei. Lossless acceleration for seq2seq generation with aggressive decoding. *arXiv preprint arXiv:2205.10350*, 2022.

Priya Goyal, Piotr Dollár, Ross Girshick, Pieter Noordhuis, Lukasz Wesolowski, Aapo Kyrola, Andrew Tulloch, Yangqing Jia, and Kaiming He. Accurate, large minibatch sgd: Training imagenet in 1 hour. *arXiv preprint arXiv:1706.02677*, 2017.

Alex Graves and Alex Graves. Long short-term memory. *Supervised sequence labelling with recurrent neural networks*, pp. 37–45, 2012.

Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. *arXiv preprint arXiv:2009.03300*, 2020.

Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015.

Sebastian Jaszczur, Aakanksha Chowdhery, Afroz Mohiuddin, Lukasz Kaiser, Wojciech Gajewski, Henryk Michalewski, and Jonni Kanerva. Sparse is enough in scaling transformers. *Advances in Neural Information Processing Systems*, 34:9895–9907, 2021.

Nikita Kitaev, Łukasz Kaiser, and Anselm Levskaya. Reformer: The efficient transformer. *arXiv preprint arXiv:2001.04451*, 2020.

Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning*, pp. 19274–19286. PMLR, 2023.

Bingbing Li, Zhenglun Kong, Tianyun Zhang, Ji Li, Zhengang Li, Hang Liu, and Caiwen Ding. Efficient transformer-based large scale language representations using hardware-friendly block structured pruning. *arXiv preprint arXiv:2009.08065*, 2020.

Jian Liu, Leyang Cui, Hanmeng Liu, Dandan Huang, Yile Wang, and Yue Zhang. Logiqa: A challenge dataset for machine reading comprehension with logical reasoning. *arXiv preprint arXiv:2007.08124*, 2020.

Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, Zeyu Wang, Rae Ying Yee Wong, Zhuoming Chen, Daiyaan Arfeen, Reyna Abhyankar, and Zhihao Jia. Specinfer: Accelerating generative llm serving with speculative inference and token tree verification. *arXiv preprint arXiv:2305.09781*, 2023.

OpenAI. Gpt-4 technical report, 2023.

Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Jonathan Heek, Kefan Xiao, Shivani Agrawal, and Jeff Dean. Efficiently scaling transformer inference. *Proceedings of Machine Learning and Systems*, 5, 2023.

Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever. Improving language understanding with unsupervised learning. 2018.

Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *The Journal of Machine Learning Research*, 21(1):5485–5551, 2020.

Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. Zero: Memory optimizations toward training trillion parameter models. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*, pp. 1–16. IEEE, 2020.

Aurko Roy, Mohammad Saffar, Ashish Vaswani, and David Grangier. Efficient content-based sparse attention with routing transformers. *Transactions of the Association for Computational Linguistics*, 9:53–68, 2021.

Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. Winogrande: An adversarial winograd schema challenge at scale. *Communications of the ACM*, 64(9):99–106, 2021.

Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. Distilbert, a distilled version of bert: smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108*, 2019.

Roy Schwartz, Gabriel Stanovsky, Swabha Swayamdipta, Jesse Dodge, and Noah A Smith. The right tool for the job: Matching model and instance complexities. *arXiv preprint arXiv:2004.07453*, 2020.

Noam Shazeer. Fast transformer decoding: One write-head is all you need. *arXiv preprint arXiv:1911.02150*, 2019.

Noam Shazeer. Glu variants improve transformer. *arXiv preprint arXiv:2002.05202*, 2020.

Sheng Shen, Zhen Dong, Jiayu Ye, Linjian Ma, Zhewei Yao, Amir Gholami, Michael W Mahoney, and Kurt Keutzer. Q-bert: Hessian based ultra low precision quantization of bert. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pp. 8815–8821, 2020.

Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.

Shaden Smith, Mostofa Patwary, Brandon Norick, Patrick LeGresley, Samyam Rajbhandari, Jared Casper, Zhun Liu, Shrimai Prabhumoye, George Zerveas, Vijay Korthikanti, et al. Using deepspeed and megatron to train megatron-turing nlg 530b, a large-scale generative language model. *arXiv preprint arXiv:2201.11990*, 2022.

Mitchell Stern, Noam Shazeer, and Jakob Uszkoreit. Blockwise parallel decoding for deep autoregressive models. *Advances in Neural Information Processing Systems*, 31, 2018.

Shane Storks, Qiaozi Gao, and Joyce Y Chai. Recent advances in natural language inference: A survey of benchmarks, resources, and approaches. *arXiv preprint arXiv:1904.01172*, 2019.

Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *arXiv preprint arXiv:2104.09864*, 2021.

Pedro Javier Ortiz Suárez, Benoît Sagot, and Laurent Romary. Asynchronous pipeline for processing huge corpora on medium to low resource infrastructures. In *7th Workshop on the Challenges in the Management of Large Corpora (CMLC-7)*. Leibniz-Institut für Deutsche Sprache, 2019.

Sainbayar Sukhbaatar, Edouard Grave, Piotr Bojanowski, and Armand Joulin. Adaptive attention span in transformers. *arXiv preprint arXiv:1905.07799*, 2019.

Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Ł ukasz Kaiser, and Illia Polosukhin. Attention is all you need. In I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett (eds.), *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017. URL https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf.

Qiang Wang, Bei Li, Tong Xiao, Jingbo Zhu, Changliang Li, Derek F. Wong, and Lidia S. Chao. Learning deep transformer models for machine translation. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pp. 1810–1822, Florence, Italy, July 2019. Association for Computational Linguistics. doi: 10.18653/v1/P19-1176. URL https://aclanthology.org/P19-1176.

Johannes Welbl, Nelson F Liu, and Matt Gardner. Crowdsourcing multiple choice science questions. *arXiv preprint arXiv:1707.06209*, 2017.

Tong Xiao, Yinqiao Li, Jingbo Zhu, Zhengtao Yu, and Tongran Liu. Sharing attention weights for fast transformer. *arXiv preprint arXiv:1906.11024*, 2019.

Ofir Zafrir, Guy Boudoukh, Peter Izsak, and Moshe Wasserblat. Q8bert: Quantized 8bit bert. In *2019 Fifth Workshop on Energy Efficient Machine Learning and Cognitive Computing-NeurIPS Edition (EMC2-NIPS)*, pp. 36–39. IEEE, 2019.

Jiali Zeng, Shuangzhi Wu, Yongjing Yin, Yufan Jiang, and Mu Li. Recurrent attention for neural machine translation. In *Proceedings of the 2021 conference on empirical methods in natural language processing*, pp. 3216–3225, 2021.

Biao Zhang and Rico Sennrich. Root mean square layer normalization. *Advances in Neural Information Processing Systems*, 32, 2019.

Susan Zhang, Stephen Roller, Naman Goyal, Mikel Artetxe, Moya Chen, Shuohui Chen, Christopher Dewan, Mona Diab, Xian Li, Xi Victoria Lin, et al. Opt: Open pre-trained transformer language models. *arXiv preprint arXiv:2205.01068*, 2022.

Aojun Zhou, Yukun Ma, Junnan Zhu, Jianbo Liu, Zhijie Zhang, Kun Yuan, Wenxiu Sun, and Hongsheng Li. Learning n: m fine-grained structured sparse neural networks from scratch. *arXiv preprint arXiv:2102.04010*, 2021.

A APPENDIX

Pre-training Hyperparameters	**1.3B**
Number of layers	24
Hidden Size	2048
FFN inner hidden size	5504
Attention heads	32
Attention head size	64
Embedding Size	2048
Warmup steps	1.5k
Learning Rate	2e-4
Adam ϵ	1e-8
Adam β_1	0.9
Adam β_2	0.95
Attention Dropout	0.0
Dropout	0.0
Weight Decay	0.01
Max Sequence Length	2048
Batch Size	1280
Train Steps	140k
RmsNorm eps	1e-06

Table 4: **The pre-training hyperparameters**.

		OPT-1.3B	Pythia-1.4B	GPT-Neo	LLaMA-ours	RecycleGPT-std	RecycleGPT-rec
Abstract Algebra	STEM	0.29	0.25	0.25	0.34	0.28	0.28
Anatomy	STEM	0.2815	0.3185	0.2074	0.3556	0.3259	0.3259
Astronomy	STEM	0.2039	0.25	0.1908	0.3421	0.2632	0.25
Business Ethics	Other	0.22	0.23	0.29	0.38	0.25	0.23
Clinical Knowledge	Other	0.2453	0.2528	0.2642	0.3245	0.2264	0.2189
College Biology	STEM	0.2569	0.3056	0.2569	0.3611	0.2431	0.2569
College Chemistry	STEM	0.17	0.21	0.22	0.39	0.27	0.24
College Computer Science	STEM	0.26	0.27	0.25	0.28	0.32	0.37
College Mathematics	STEM	0.23	0.24	0.28	0.3	0.3	0.28
College Medicine	Other	0.2543	0.2543	0.2428	0.3121	0.2312	0.1908
College Physics	STEM	0.2843	0.2451	0.2255	0.2353	0.2549	0.2843
Computer Security	STEM	0.18	0.22	0.28	0.46	0.3	0.3
Conceptual Physics	STEM	0.2213	0.3106	0.2596	0.366	0.2085	0.2511
Econometrics	Social Science	0.2719	0.2281	0.2632	0.2632	0.2456	0.2281
Electrical Engineering	STEM	0.3034	0.2621	0.2552	0.2483	0.2414	0.2483
Elementary Mathematics	STEM	0.254	0.2672	0.2937	0.2619	0.2487	0.2646
Formal Logic	Humanities	0.1349	0.127	0.1825	0.2063	0.1508	0.1508
Global Facts	Other	0.35	0.36	0.2	0.31	0.32	0.31
High School Biology	STEM	0.2194	0.2548	0.2484	0.3613	0.2581	0.2613
High School Chemistry	STEM	0.2709	0.2512	0.2414	0.2808	0.3005	0.2562
High School Computer Science	STEM	0.34	0.27	0.35	0.31	0.3	0.32
High School European History	Humanities	0.2303	0.2545	0.2303	0.4485	0.2485	0.2909
High School Geography	Social Science	0.2525	0.2424	0.3283	0.3535	0.2576	0.2576
High School Government And Politics	Social Science	0.2694	0.1917	0.2591	0.4508	0.2642	0.2539
High School Macroeconomics	Social Science	0.2538	0.2128	0.3487	0.3436	0.2103	0.2103
High School Mathematics	STEM	0.237	0.2444	0.2481	0.2407	0.2556	0.2556
High School Microeconomics	Social Science	0.1975	0.2269	0.2395	0.3319	0.2185	0.2185
High School Physics	STEM	0.2848	0.245	0.2384	0.2781	0.2848	0.2914
High School Psychology	Social Science	0.244	0.2569	0.3064	0.4624	0.2294	0.2477
High School Statistics	STEM	0.2639	0.2454	0.4074	0.3519	0.412	0.3333
High School Us History	Humanities	0.1814	0.2843	0.201	0.3578	0.2549	0.2451
High School World History	Humanities	0.2405	0.2574	0.2194	0.4388	0.2785	0.2489
Humanities Aging	Other	0.2646	0.3274	0.1839	0.417	0.287	0.278
Humanities Sexuality	Social Science	0.2366	0.2519	0.2748	0.3817	0.2748	0.2214
International Law	Humanities	0.2727	0.3554	0.2314	0.5537	0.3802	0.3719
Jurisprudence	Humanities	0.2222	0.25	0.2963	0.4352	0.2685	0.2222
Logical Fallacies	Humanities	0.2515	0.3006	0.2577	0.4233	0.3129	0.3067
Machine Learning	STEM	0.2321	0.2054	0.1696	0.2411	0.2946	0.25
Management	Other	0.1553	0.2524	0.2718	0.3592	0.2039	0.165
Marketing	Other	0.235	0.2607	0.265	0.4615	0.265	0.2863
Medical Genetics	Other	0.27	0.26	0.29	0.4	0.26	0.23
Miscellaneous	Other	0.2746	0.2746	0.2363	0.4317	0.2682	0.2695
Moral Disputes	Humanities	0.2341	0.2775	0.2457	0.3815	0.2601	0.2341
Moral Scenarios	Humanities	0.2447	0.248	0.2704	0.2425	0.2458	0.2514
Nutrition	Other	0.2745	0.2582	0.317	0.3922	0.2614	0.2843
Philosophy	Humanities	0.1961	0.2926	0.3248	0.4116	0.299	0.299
Prehistory	Humanities	0.2747	0.2562	0.3056	0.3488	0.2407	0.2778
Professional Accounting	Other	0.2553	0.266	0.2553	0.2766	0.2553	0.2695
Professional Law	Humanities	0.2269	0.2627	0.2477	0.2934	0.2451	0.2471
Professional Medicine	Other	0.375	0.1838	0.4301	0.4449	0.1765	0.2426
Professional Psychology	Social Science	0.268	0.2729	0.2696	0.3578	0.2516	0.268
Public Relations	Social Science	0.1727	0.3091	0.1818	0.3727	0.2	0.2364
Security Studies	Social Science	0.2041	0.2041	0.2939	0.3388	0.2327	0.2286
Sociology	Social Science	0.2338	0.2637	0.2289	0.4726	0.2537	0.2637
Us Foreign Policy	Social Science	0.2	0.26	0.31	0.45	0.24	0.24
Virology	Other	0.2771	0.2771	0.3193	0.3193	0.247	0.247
World Religions	Humanities	0.2573	0.2924	0.2807	0.4795	0.2573	0.2749

Table 5: **Detailed five-shot results per domain on the MMLU test sets.**